

Demand Paging in xv6

Lee Ruo Chen 2500530¹

¹*Singapore Institution of Technology, Singapore*

1. Introduction

The standard xv6 operating system employs eager memory allocation during process execution. When a program is loaded or requests memory, the kernel immediately allocates physical frames for the entire requested logical address space. Consequently, programs exceeding available physical memory cannot execute, and valuable memory is often wasted on code or data that the process rarely, if ever, accesses.

Demand paging resolves this inefficiency. Rather than loading a process in its entirety, a demand paging system dynamically brings individual pages into memory only when they are explicitly needed. This lazy approach optimizes system performance by drastically reducing the initial physical memory footprint, enabling faster process startup times, and allowing the OS to execute programs whose virtual footprint far exceeds the actual physical memory.

The mechanism of demand paging relies on the cooperation between the CPU and the operating system. When a process attempts to access an unmapped page, the CPU triggers a page fault, trapping into the kernel. The operating system catches this trap, locates the required data, and allocates a free physical frame. It then updates the process's page table, setting the Invalid/Valid bit to valid to indicate the page is now in memory and instructs the CPU to restart the faulting instruction, the process resumes and executes the previous failed instruction successfully.

Implementing demand paging in xv6 dramatically overcomes the strict limitations of eager allocation. This paper outlines a comprehensive architectural blueprint for adding a robust demand paging subsystem to xv6. The proposed design focuses on three core pillars: Pure Demand Paging for lazy memory allocation, the Enhanced Second Chance algorithm for intelligent page replacement, and a dedicated raw disk partition to safely manage swap space.

2. Swap Space Architecture

When it comes to demand paging, a swap partition is needed in order to store swapped pages that get unallocated by the page replacement algorithm. A naive approach to implementing this in xv6 would be to create a hidden file to store swapped out pages. Utilizing the standard xv6 file system for demand paging introduces critical vulnerabilities regarding kernel deadlock and severe performance overhead.

* Demand Paging in xv6 by Lee Ruo Chen, Singapore Institution of Technology

✉ 2500530@sit.singaporetech.edu.sg (L. R. C. 2500530)

The xv6 file system utilizes sleeping locks and a transaction log that forces all file system operations to be recorded prior to disk execution to prevent data loss. This creates a fatal edge case: if a process suffers a page fault in the middle of an active file system transaction, and physical memory is full, the kernel must swap out a page to resolve the fault. If the memory manager attempts to write this evicted page to a standard swap file, it initiates a new file system operation. The logging system may then put the page fault handler to sleep, forcing it to wait for the currently active transaction to commit. Because the active transaction belongs to the very process that is suspended by the page fault, a circular wait occurs. The process waits for the handler to bring the page into memory, but the handler is sleeping, waiting for the process to release the log locks, resulting in a permanent kernel deadlock.

To bypass the file system entirely, this design proposes a dedicated raw swap partition. By modifying *mkfs.c* and the system *Makefile*, a continuous block of raw disk sectors can be reserved at the end of the fs.img disk image. The memory management subsystem can then use low-level buffer cache operations (*bread()* and *bwrite()*) to interact directly with the disk controller. This raw partition approach guarantees that memory eviction operates independently of file system locks, ensuring system stability while maximizing swap I/O performance.

3. Pure Demand Paging: Lazy Allocation

When it comes to implementing demand paging, we can either choose to apply pure demand paging, where a page is never brought into memory until required by the process, or apply a working-set model that helps the operating system predict which set of pages a program needs by incorporating a sliding window to look at the program's history of used pages. For this implementation, pure demand paging is utilized to minimize initial memory overhead.

Implementing pure demand paging in xv6 requires fundamental architectural changes to how the kernel handles process memory growth and hardware traps.

3.1. Delaying Physical Allocation

In the default architecture of xv6, programs requesting for additional heap memory will be allocated with *sbrk()*, which eagerly allocates physical memory and maps the corresponding page table entries entirely. This function works by calling *growproc()*, which uses *uvmmalloc()* to immediately find free physical frames with *kalloc()* to map them with *mappages()*. To implement lazy allocation, *sbrk()* must be modified to decouple virtual memory from physical memory allocation. When a process requests *n* bytes, the system call will simply increment the process's internal size tracker (*myproc()->sz*) and return to the user. Zero physical frames are allocated, and the page table remains unchanged.

To maintain flexibility, the *sbrk()* system call can be modified to accept a flag to indicate the allocation type. For demand paging, the system call will divert from the standard *growproc()* memory allocation. Instead, it will perform an overflow safety check and increment the process's size attribute, leaving the actual frame allocation to the trap handler. This allows eager allocation to still be an option.

3.2. Catching and Resolving Page Faults

With lazy allocation, the physical memory is never allocated, the moment the user process attempts to read or write to the newly requested heap space, the Memory Management Unit will fail the address translation as the current page table is empty. This page fault triggers a trap into the kernel, specifically routing to the *usertrap()* handler in *trap.c*.

The *usertrap()* function must be modified to bring pages into the physical memory when a page fault occurs. It identifies the page fault by reading the RISC-V cause register (*r_scause()*), looking specifically for Load (13) or Store (15) page faults. The handler retrieves the faulting virtual address from the *r_stval()* register and verifies that the address is within the boundaries of the process's allocated size. Once validated, the kernel will call *kalloc()* to find a free physical frame. If *kalloc()* succeeds, the frame will be sanitized with *memset* to prevent data leakage, then *mappages()* is executed to map the page to the new physical frame with the appropriate permissions. Finally, the kernel instructs the CPU to restart the faulting instruction, allowing the user process to proceed seamlessly with the page now allocated in memory.

4. Page Replacement: Enhanced Second Chance

When it comes to physical memory, it is always limited and in xv6, the physical memory is limited to 2GB[3]. Consequently, the pool of free physical frames managed by *kalloc()* will eventually be exhausted. When an allocation request occurs under full memory, a page replacement algorithm is required to select a victim page to evict to the raw swap partition. For this implementation, the Enhanced Second Chance algorithm is utilized because it provides a highly efficient, hardware assisted approach to victim selection.

The algorithm relies on status bits within the Page Table Entry. In xv6, the page table entries have status bits to indicate the state of the page, we can integrate an Access (*PTE_A*) and Dirty (*PTE_D*) bit to determine if a page has been recently accessed or modified. This complements perfectly with the enhanced second chance algorithm where it tracks the reference and dirty bits to decide victim pages. It categorizes pages into four classes to find the optimal eviction victim:

Class 1: (0,0), Not accessed recently and not modified. The best victim.

Class 2: (0,1), Not accessed but dirty, still a good victim, but requires a disk write.

Class 3: (1,0), Accessed recently but not dirty, used recently so page should be kept.

Class 4: (1,1), The worst victim, both accessed recently and dirty.

To implement this, the memory manager utilizes a pointer that analyzes the data structure that contains all allocated physical frames. As the pointer iterates through the data structure, it searches for the lowest class victim. If it finds a recently accessed page (*PTE_A == 1*), it grants the page a "second chance" by clearing the *PTE_A* bit to 0 and advancing. If the page remains untouched by the time the pointer completes a full iteration, it will be dropped to with class 1 or 2 and will be evicted.

Once a victim frame is selected, the kernel checks its dirty bit. If a page is dirty, it is needed to copy the frame with the *bwrite()* function into a free sector in the dedicated raw swap partition. This allows the page to be retrieved again with the modified data back into the memory if needed by a process.

To track the physical location of the swapped data without introducing a new data structure, the format of the page table entry can be modified. When a page is evicted, the kernel clears the Valid bit. Since it is invalid, the Memory Management Unit completely ignores the entry. A new custom bit can be set (*PTE_S*) to indicate the page is swapped, the kernel overwrites the Physical Page Number field of the entry with the exact disk sector location where the data was stored. When the process subsequently page faults for the swapped page, *usertrap()* simply reads the sector location from the invalid entry, allocates a new frame, and reads the data back into memory.

5. Performance Optimization

While pure demand paging excels at initial memory overhead, it introduces a severe performance vulnerability known as thrashing. Thrashing occurs when a process lacks sufficient physical frames required to execute its code, resulting in a continuous, paralyzing cycle of page faults and disk I/O. Under a strict pure demand paging model, this bottleneck occurs during context switches. Because a resumed process will always start with zero pages in memory, to even execute its first line of code, it might need many different pages. In the event that the system is low on physical memory, the process might continuously page fault to get different pages to execute. This creates a massive bottleneck where the process is thrashing before it even gets a chance to work.

To neutralize this threat, the memory manager can be enhanced to track a process's working set which is defined as the subset of pages actively referenced over a sliding window. By repurposing the same *PTE_A* access bits used in the page replacement algorithm, the operating system can construct a lightweight data structure that consists of the historical use of pages per process. In xv6, process scheduling is driven by the round robin scheduling that routinely pauses the active process with *yield()*. This timer interval provides an opportunity to perform a sweep across recently accessed pages and record it in an array.

To execute this sweep before *yield()* is called, we need to implement a way to traverse the page table in xv6. In xv6, one of the ways to retrieve a page table entry is to use its *walk()* function, which returns a page table entry corresponding to the given virtual address. If we were to use this function to traverse through the whole page table, it would be incredibly inefficient. Hence, we need to create a custom traversal function that acts similar to xv6's *freewalk()*[4] or *uvmunmap()*. These functions are custom recursive functions that visit every valid entry in the page table tree.

The recursive routine begins at the process's root page directory and steps through the 3-level RISC-V Sv39 page table tree. As it loops through all the entities in the directory, it checks for validity. If the valid entry found lacks read, write, or execute permissions, it acts as a pointer to the next level of the tree. The function calls itself recursively to dive down into the next directory. Once a valid leaf entry is found with standard permissions, it represents a mapped physical page. If the entry's access bit is set to *PTE_A == 1*, the corresponding virtual address is recorded in a process specific fixed working-set array which limits the number of pages allowed within *struct proc*.

Crucially, the kernel then clears the *PTE_A* bit back to 0. If the bit were not cleared, the

array would permanently accumulate every page the process ever touched. By clearing it, the hardware is forced to re-flag only the pages utilized during the subsequent execution slice, creating a highly accurate, sliding-window approximation of the process's immediate memory needs.

This historical tracking enables the operating system to shift from pure demand paging to prepaging during context switches with a working set. When the scheduler selects a suspended process to run, the memory manager intercepts the context switch. Instead of waiting for the process to blindly page fault, the kernel consults the process's working-set array. The operating system proactively allocates frames and issues *bread()* requests to pull the entire cluster of pages from the raw swap partition into memory. By prepagging the working set, the process resumes with its required memory already resident, bypassing the series of page faults that occur during startup and effectively neutralizing the threat of thrashing.

6. Conclusion

This report has outlined a comprehensive architectural blueprint for integrating a demand paging subsystem into the xv6 operating system. By transitioning away from default eager allocation, xv6 can leverage pure demand paging to drastically reduce initial memory overhead and support programs larger than physical memory. This lazy allocation is supported by a dedicated raw disk partition that safely bypasses file system deadlocks, and the hardware assisted Enhanced Second Chance algorithm, which efficiently manages physical frame eviction under heavy memory pressure.

Furthermore, while pure demand paging introduces the risk of thrashing, approximating a process's working set via recursive page table sweeps allows the kernel to proactively prepage memory during context switches, ensuring seamless execution. Ultimately, this design successfully modernizes xv6's memory management. It grants user processes the ability to have sizes not constrained to the limits of the physical memory, higher degree of multiprogramming, decreased I/O overhead, and physical memory efficiency.

References

- [1] R. Cox, F. Kaashoek, and R. Morris, xv6: a simple, Unix-like teaching operating system, Revision 5. Cambridge, MA: Massachusetts Institute of Technology, 2025. [Online]. Available: <https://pdos.csail.mit.edu/6.828/2025/xv6/book-riscv-rev5.pdf>. [Accessed: Apr. 1, 2026].
- [2] GeeksforGeeks, "What is Demand Paging in Operating System?," GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org/operating-systems/what-is-demand-paging-in-operating-system/>.
- [3] M. Vutukuru, "Notes on xv6 memory management," Indian Institute of Technology (IIT) Bombay. [Online]. Available: <https://www.cse.iitb.ac.in/~mythili/os/notes/old-xv6/xv6-memory.pdf>.
- [4] CS 326: Operating Systems, "Page Tables," University of San Francisco (USFCA), Spring 2025. [Online]. Available: <https://cs326-s25.cs.usfca.edu/guides/page-tables>.